

Q: Any questions or
comments?

Distributed Systems & Networking (part 2, inc. TCP)

COMP2014 Distributed Systems

Chris Greenhalgh

School of Computer Science

Notices to share with students at the start of an EchoPoll session

The following notices are taken from the
[‘Code of Practice for the Safe Use of EchoPoll
v1.1’](#)

While text responses may sometimes appear
to be anonymous to the audience, they are
always traceable

Please be advised that names and e-mail
addresses of respondents may be displayed
on screen

Contents

- Java TCP Programming
- Layer 1 / 2 Networks (revision)
- Routing and Configuration (revision)
- Java Data Encoding

5	Application
4	Transport (TCP, UDP)
3	Internet (IP)
2	Network interface / Link
1	Physical

Java TCP programming

Java TCP API

(in java.net)

- **ServerSocket** represents a server TCP socket 😊
 - Bound to a particular port, waiting to **accept** connection requests from client sockets...
- **Socket** represents one end of a TCP connection
 - Explicitly created by a client with a specified server IP address and port number
 - Has an `java.io.InputStream` to receive bytes from and a `java.io.OutputStream` to send bytes to
 - Also returned from `accept` in server
 - Client and server are equivalent after initial connection
 - Can be queried for local and remote addresses and port numbers

You don't need to memorise the API, but you should understand the basic patterns of use. It is the same as in C/C++, python, etc.

TCP client: makes connection to server, sends request and receives reply

```
import java.net.*;
import java.io.*;
public class TCPClient {
    public static void main (String args[]) {
        // arguments supply message and hostname of destination
        Socket s = null;
        try{
            int serverPort = 7896;
            s = new Socket(args[1], serverPort);
            DataInputStream in = new DataInputStream( s.getInputStream() );
            DataOutputStream out =
                new DataOutputStream( s.getOutputStream() );
            out.writeUTF(args[0]); // see encoding and representation
            String data = in.readUTF();
            System.out.println("Received: "+ data) ;
        } catch (UnknownHostException e) {
            System.out.println("Sock:"+e.getMessage());
        } catch (EOFException e) {System.out.println("EOF:"+e.getMessage());}
        } catch (IOException e) {System.out.println("IO:"+e.getMessage());}
        } finally {if(s!=null) try {s.close();} catch (IOException e)
        {System.out.println("close:"+e.getMessage());}}
    }
}
```

1. **Make a Socket -> connect**
(Unknown host / IO exception -> problem)
2. **(Prepare input stream = from server)**
3. **(Prepare output stream = to server)**
4. **Write request bytes to output stream**
5. **Read response bytes from input stream**
(EOF -> not enough input)
6. **...**
7. **Close socket -> terminate connection**

TCP server: makes a connection for each client and then echoes the client's request

1. **Make a ServerSocket with known port**
2. **(repeatedly...)**
3. **.accept a new client connection represented by a new Socket**
4. **... (make a thread to handle that client)**

```
import java.net.*;
import java.io.*;
public class TCPServer {
    public static void main (String args[]) {
        try{
            int serverPort = 7896;
            ServerSocket listenSocket = new ServerSocket(serverPort);
            while(true) {
                Socket clientSocket = listenSocket.accept();
                Connection c = new Connection(clientSocket);
            }
        } catch(IOException e) {System.out.println("Listen :"+e.getMessage());}
    }
}
```

// this figure continues on the next slide

TCP Server, *continued*

One instance per client:

1. **(Prepare input stream = from client)**
2. **(Prepare output stream = to client)**
3. **(start client-specific thread)**
4. **Read request bytes from input stream**
5. **...**
6. **Write response bytes to output stream**
7. **(EOF -> not enough input)**
8. **Close socket -> terminate connection**
9. **(terminate thread)**

```
class Connection extends Thread {
    DataInputStream in;
    DataOutputStream out;
    Socket clientSocket;
    public Connection (Socket aClientSocket) {
        try {
            clientSocket = aClientSocket;
            in = new DataInputStream( clientSocket.getInputStream() );
            out = new DataOutputStream( clientSocket.getOutputStream() );
            this.start();
        } catch(IOException e) {System.out.println("Connection:"+e.getMessage());}
    }
    public void run(){
        try {
            // an echo server
            String data = in.readUTF();
            out.writeUTF(data);
        } catch(EOFException e) {System.out.println("EOF:"+e.getMessage());}
        } catch(IOException e) {System.out.println("IO:"+e.getMessage());}
        } finally{ try {clientSocket.close();}catch (IOException e){/*close failed*/}}
    }
}
```

TCP Considerations

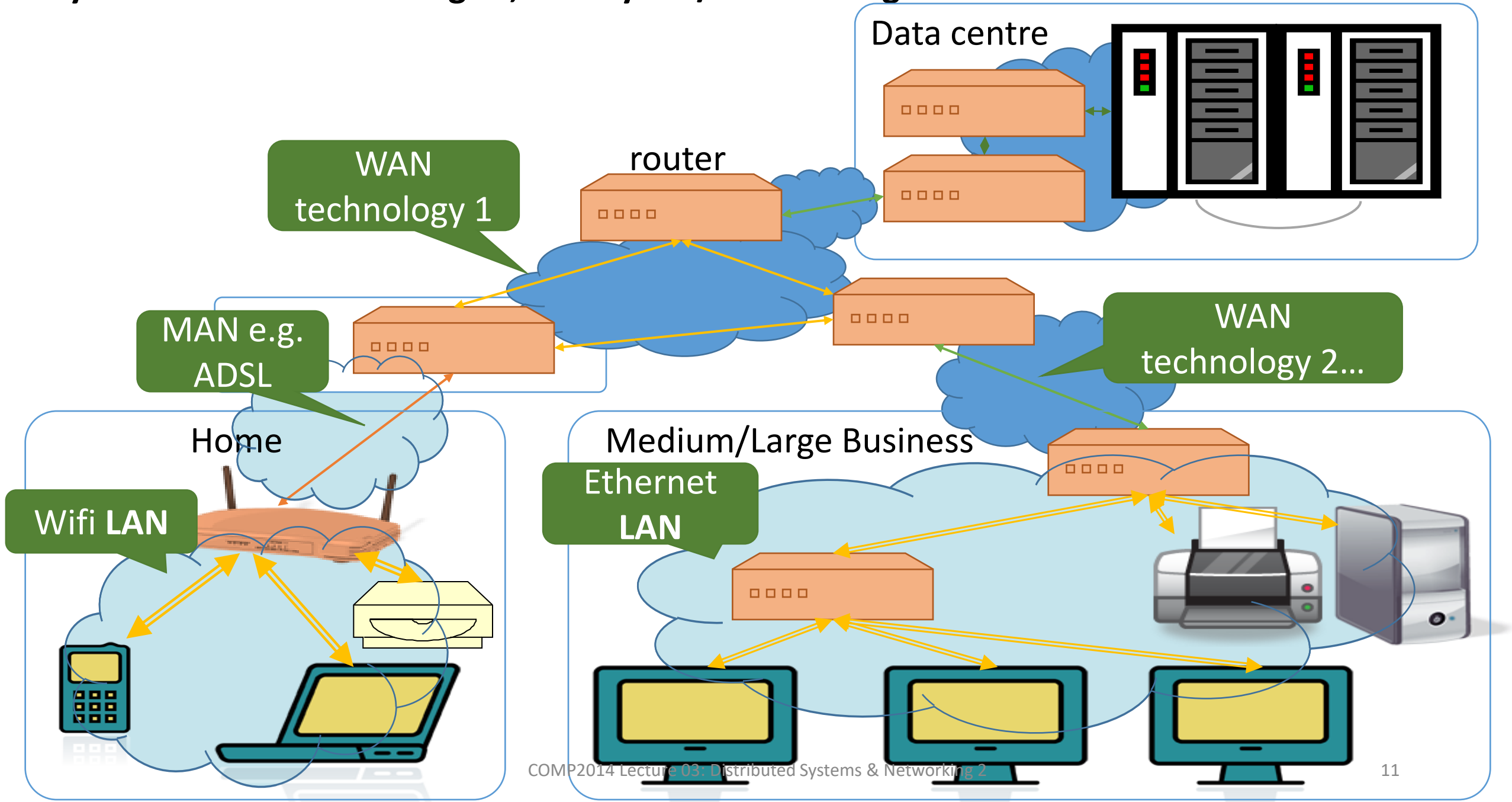
- **Message size** – application reads/writes any number of **bytes** at a time until the connection is **closed**
 - application does NOT see any message boundaries, so needs to communicate any message boundaries explicitly in the bytes sent
- **Message destinations** – a server can accept connection requests from any client
 - Once the connection is established then bytes are communicated only between that pair of processes
- **Blocking** - a limited amount of data is buffered at sender and receiver sockets and sends will **block** once the sender buffer is full (known as **Flow control**)
 - Connecting to a server, accepting a connection and reading (receiving) from a socket normally block – these are typically done in a dedicated thread in Java

TCP Considerations (cont.)

- **Failure model** – communication is reliable short of complete connection failure
 - **Lost messages** are automatically **retransmitted**
 - **duplicate** packets are discarded and **out-of-order** packets are reordered using sequence numbers
 - Bytes are read by the receiving application in exactly the same order as they are written by the sending application
 - If a loss is not resolved within a time limit (often 30 seconds) then whole connection fails
 - It isn't possible to know whether the last data got through
 - (See also later lecture on reliable communication)
- **Uses of TCP**
 - Almost anything needed to transfer data reliably and/or in large amounts
 - E.g. HTTP, SSH (secure shell), FTP (file transfer), SMTP (email)

Layer 1/2 Networks

Physical Network Technologies, i.e. Layer 1/2 Technologies



Layer 1 (Physical) / layer 2 (Network interface / Link)

- Each physical network can use a different technology with different performance characteristics (see next slide)
- One computer can connect to several different networks
 - E.g. a phone on WiFi and 3G, a server on two different Ethernets
- Each network interface typically has its own layer 2 network address, often called a **MAC address**
 - (MAC = **M**edium **A**ccess **C**ontrol)
- Some (but not all) networks allow a single message to be physically sent to all machines on that network
 - “**Broadcast**” or more selectively “**multicast**” – see later lecture
 - E.g. Ethernet, WiFi

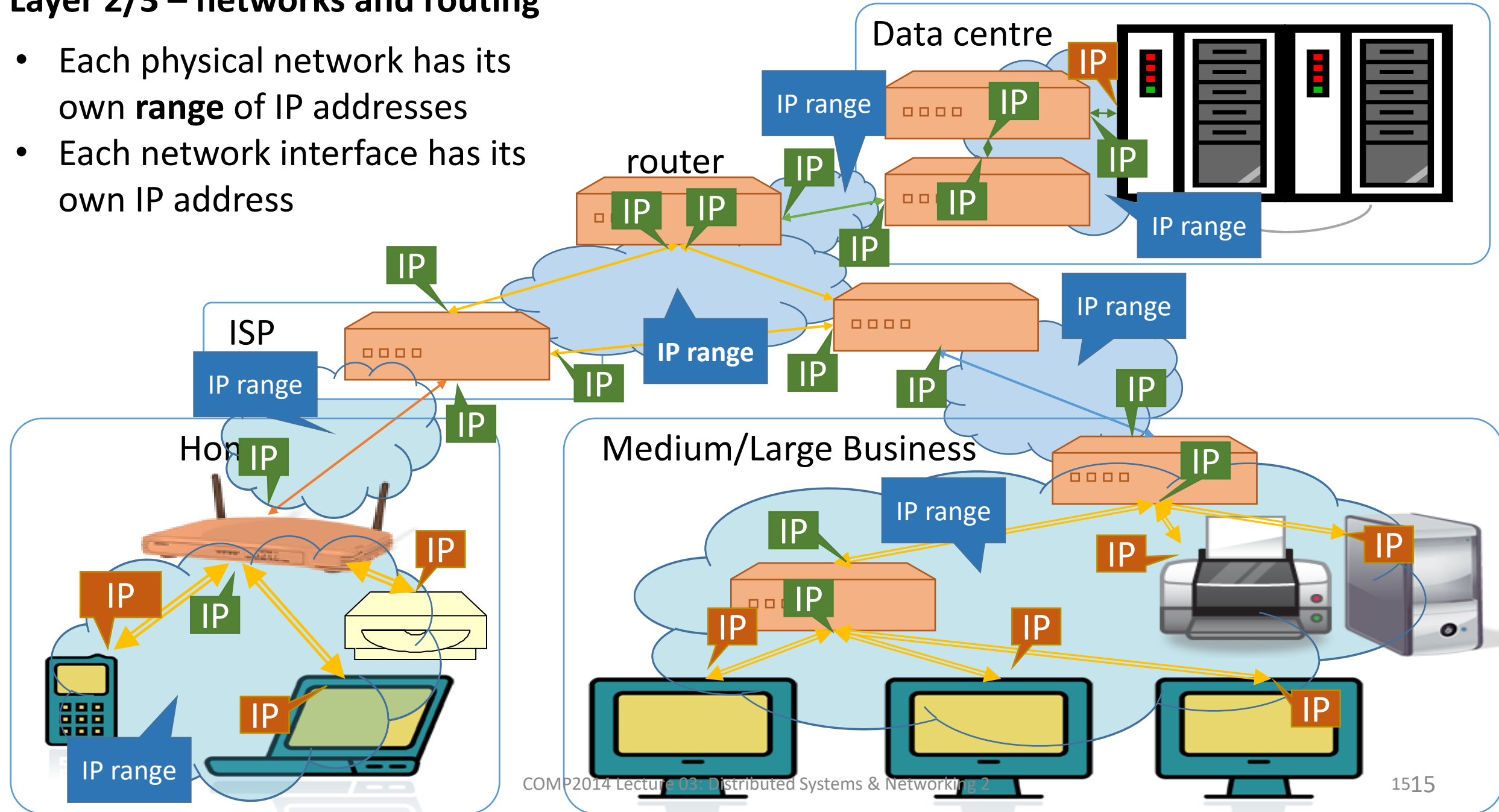
Performance of communication channels

- Communication can be realised with different paradigms and technologies, but each has:
- **Reliability** or **delivery ratio**: fraction of messages successfully delivered
 - Opposite of **packet loss ratio**
- **Latency**: the delay between the start of a messages transmission and beginning of its receipt
 - Including propagation delay (e.g. speed of light), channel access time (e.g. waiting for a busy shared network) and end system delays (e.g. waiting for a busy OS)
- **Bandwidth**: total information transmitted in a unit time
 - Typically shared by concurrent communication channels
- **Jitter**: variation in delay (i.e. variation in latency)
 - Important for multimedia and interactive data

Routing and Configuration

Layer 2/3 – networks and routing

- Each physical network has its own **range** of IP addresses
- Each network interface has its own IP address



Routing and forwarding

- **Routers** have interfaces on two or more networks and **forward** packets from one network to the next
- Host and routers have **routing tables**; these identify for each destination IP address which interface/host to send it to next
 - (As seen in lecture 1, **VMs** and **containers** also have their own routing tables, typically configured by the VM host or container system, respectively)
- Hosts and **routers** work together to deliver each packet to its destination “hop by hop” (aka store and forward)

Host Configuration

- Every host on the Internet needs to know (for its routing table)
 - Its own **IP address**
 - The IP address range on its local **network**
 - The IP address of one or more routers to send any other messages to (often called a **gateway** or default route)
 - (usually plus the IP address of a **DNS server**)
- This is often discovered when the machine connects to a local network by using **DHCP** (Dynamic Host Configuration Protocol)
 - A DHCP server on the local network provides the information – see later lecture on multicast & broadcast
- Servers may be statically (i.e. manually) configured

Router Configuration

- Routers are partly configured statically (manually), with:
 - IP address for each of its own network interfaces
 - IP address range for each **directly** connected network
 - (Other settings...)
- Routers use **routing protocols** to exchange information with other routers
 - E.g. OSPF within an organization or BGP between organisations
 - To determine what routers and links exist, what networks (address ranges) are where and if links are down
 - A routing algorithm can then build a complete routing table

Java TCP example

1 Parse arguments

e.g. *www.comp2014.com test*

Hosts, processes and steps

2 Look up IP address
for *www.comp2014.com*
DNS

4 request *test*

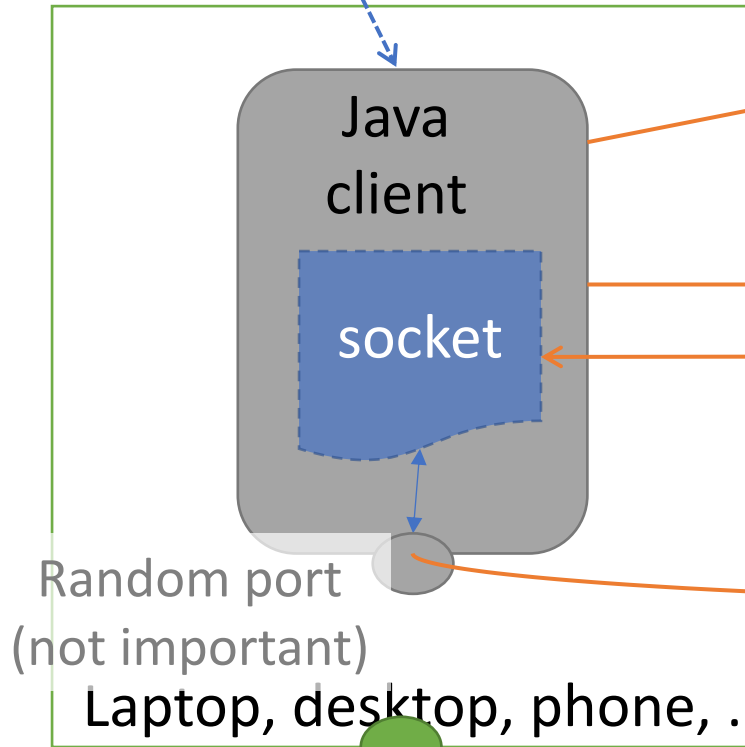
= byte stream over TCP

5 response

= byte stream over TCP

3 TCP Connect to server

IP 74.125.230.240, port 9123

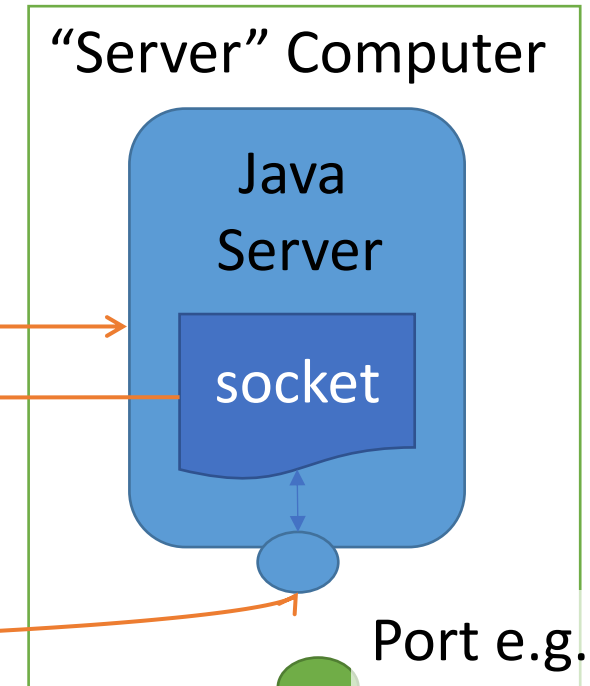


Random port
(not important)

Laptop, desktop, phone, ...

IP Address

e.g. *128.243.21.116*
(not important)



"Server" Computer

Java
Server

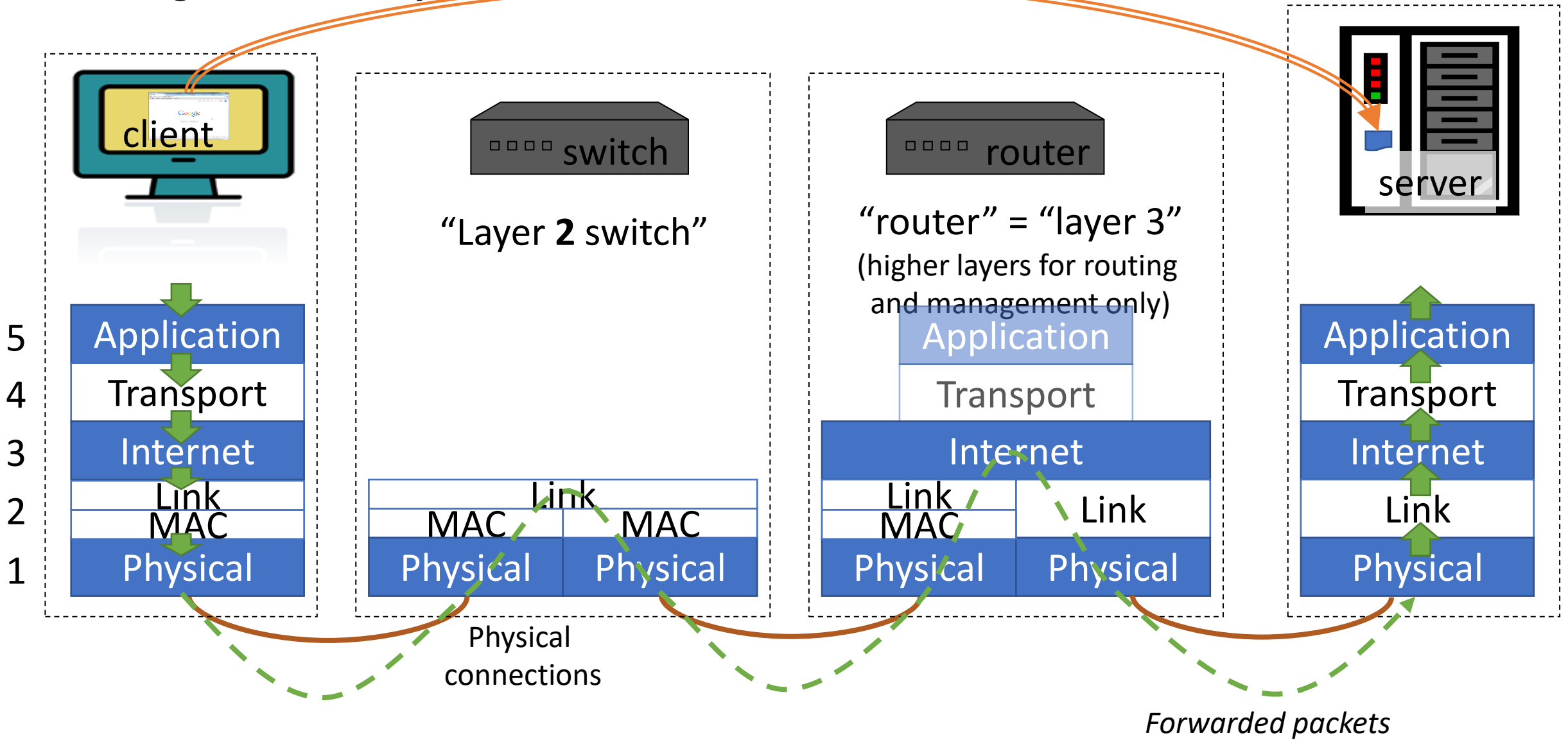
socket

Port e.g. 9123

IP Address

e.g. *74.125.230.240*,
Domain name
e.g. *www.comp2014.com*

Looking across the layers:



Note that each layer is built on top of the lower layers

Every application communication will cause one or more lower-layer messages

(Java) External Data Representation

External data representation and marshalling

- Networks (e.g. UDP, TCP) just transport streams or blocks of bytes
- Applications must agree on how data within a program will be **represented** as bytes on a network
 - E.g. what encoding of an integer, what character set for a character, how to identify an object, how to represent an array, how to represent a request, ...
- **Marshalling** is the process of converting program data to network form
 - **unmarshalling** is the converse

External data representation examples for Java

- Note: standard libraries available for marshalling
- `String.getBytes` – string → array of bytes
 - Reverse: `new String(bytes, charset)`
 - Depends on character encoding, e.g. UTF-8, US-ASCII
 - Note: how long is the string?! Whole UDP message? To end of TCP stream? Up to a marker byte? Send explicit length first? (how?)
- `DataOutputStream` / `DataInputStream`
 - Java-specific encoding of all primitive Java data types
 - Note: what data type is next?! Pre-agreed order of data types? Send explicit type marker first?

- **JSON** (JavaScript Object Notation) – cross-language UTF-8 text-based encoding including maps and arrays
 - E.g. `{"a": "hello!", "b": 123}`
 - Very widely used cross-language; relatively simple; more verbose than binary encoding, but can be compressed
- **XML** – cross-language text-based encoding with HTML-like markup
 - E.g. `<a>Hello!</a><b>123</b>`
 - More complex and verbose than JSON but can also be compressed
- Google protocol buffers –JSON/XML-like, binary
 - <https://developers.google.com/protocol-buffers/>
- HTTP – line-based US-ASCII text-based encoding plus binary body with explicit length
 - E.g.
`GET /~pszcmg/Project_ideas.html HTTP/1.1`
`Host: www.cs.nott.ac.uk`

Summary

Java TCP Programming Summary

- **TCP** provides a reliable connection-oriented bidirectional byte-based stream service
 - Persistent network packet losses result in the connection failing
 - Blocking connect, accept and receive and potentially blocking send (due to flow control) with optional timeouts
 - For almost any application transferring data reliably and/or in significant amounts
 - E.g. HTTP, SSH (secure shell), FTP (file transfer), SMTP (email)
- Java API comprises `java.net.Socket` and `java.net.ServerSocket`
 - Plus `java.net.InetAddress`, `java.io.InputStream`, `java.io.OutputStream`, etc.

Request-reply: TCP vs UDP

TCP

- Establish **connection** first
 - Request = **stream** of bytes, reply = stream of bytes
 - *Note: how many bytes?! When to stop?*
- No size limit
 - Reliably delivered in order
 - Unless connection fails
 - Flow & congestion control
- Always 1-to-1 (unicast)
 - E.g. HTTP, SMTP

UDP

- No connection overhead
 - Request = 1 **message**, reply = 1 message
- Limited to a few kB
 - *Note: fragmentation magnifies loss rate*
 - May be lost or out of order
 - Application recovers
 - No flow or congestion control
- Unicast, multicast or broadcast
 - E.g. DNS, DHCP

Low-level Networking Summary

- Network interface (Link) Layer, 2, deals with single hop (link) communication
 - E.g. 802.11 – Ethernet, WiFi
 - May have its own "**MAC**" (Medium Access Control) addresses
 - May (or may not) allow **broadcast** or **multicast**
- Each network can use a different technology, and have performance characteristics (**reliability**, **latency**, **bandwidth**, jitter)

Network Configuration Summary

- Each physical **network** has its own **range** of IP addresses
- Each **interface** on that network has its own IP address which must be in that range
- **Routers forward** packets between networks
 - Hosts and routers work together to deliver each packet to its destination “**hop** by hop”
- Hosts and routers have **routing tables** which determine where to send a packet next
 - This may be (partly or entirely) statically (i.e. manually) configured
 - On normal hosts this is often set up dynamically using **DHCP**
 - On routers it is normally built dynamically using **routing protocols**

External Data Representation Summary

- Internal process data must be transformed to (**marshalled**) and from (**unmarshalled**) an external data representation as sequences of bytes to be transmitted over the network
- E.g.
 - various string encodings (e.g. UTF-8, US-ASCII)
 - Java DataInput/OutputStream
 - **JSON**
 - XML, Google protocol buffers
 - DIY binary encodings
 - ...

Next

- Lab 2: Java TCP & UDP
- Lecture 4: (network) security